LAB ASSIGNMENT – 2.2

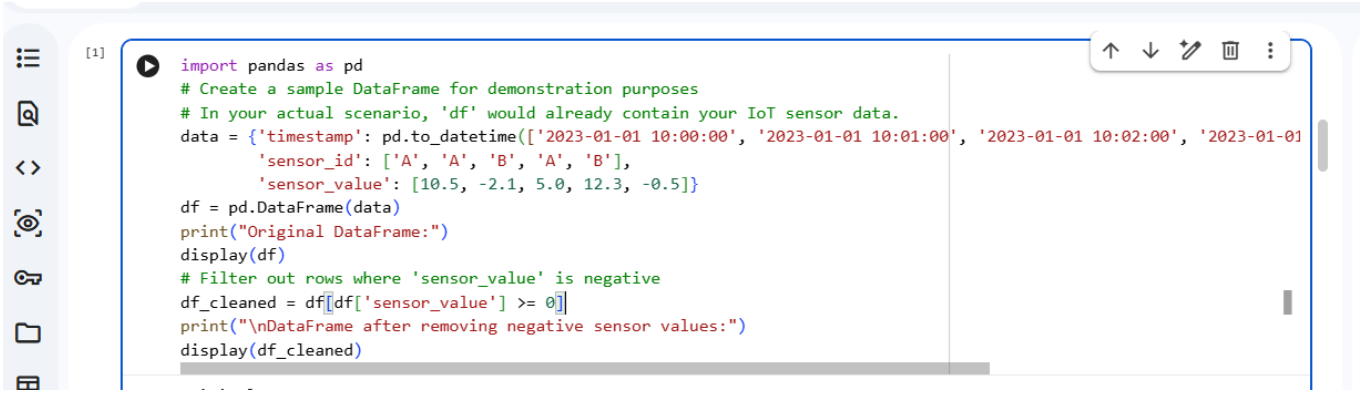
HALLTICKET N0 : 2303A51416

BATCH NO : 29

TASK - 01 : You are cleaning IoT sensor data where negative values are invalid.

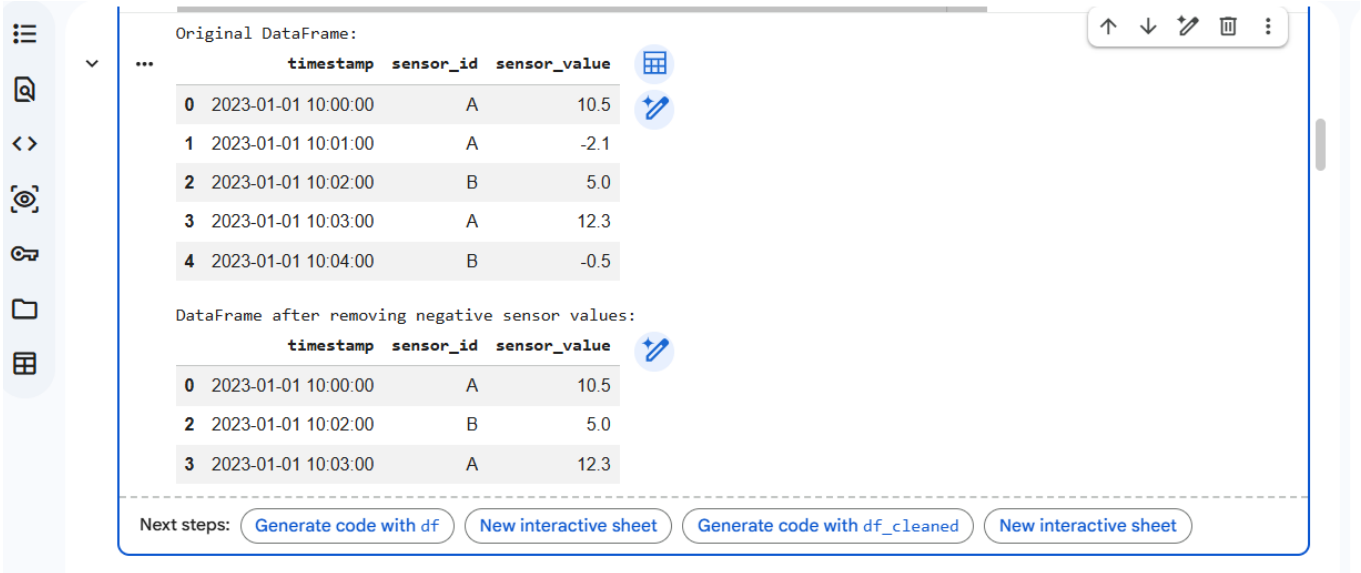
PROMPT : Use Gemini in Colab to generate a function that filters out all negative numbers from a list

CODE :



```
[1]
import pandas as pd
# Create a sample DataFrame for demonstration purposes
# In your actual scenario, 'df' would already contain your IoT sensor data.
data = {'timestamp': pd.to_datetime(['2023-01-01 10:00:00', '2023-01-01 10:01:00', '2023-01-01 10:02:00', '2023-01-01
        'sensor_id': ['A', 'A', 'B', 'A', 'B'],
        'sensor_value': [10.5, -2.1, 5.0, 12.3, -0.5]}
df = pd.DataFrame(data)
print("Original DataFrame:")
display(df)
# Filter out rows where 'sensor_value' is negative
df_cleaned = df[df['sensor_value'] >= 0]
print("\nDataFrame after removing negative sensor values:")
display(df_cleaned)
```

OUTPUT :



Original DataFrame:

	timestamp	sensor_id	sensor_value
0	2023-01-01 10:00:00	A	10.5
1	2023-01-01 10:01:00	A	-2.1
2	2023-01-01 10:02:00	B	5.0
3	2023-01-01 10:03:00	A	12.3
4	2023-01-01 10:04:00	B	-0.5

DataFrame after removing negative sensor values:

	timestamp	sensor_id	sensor_value
0	2023-01-01 10:00:00	A	10.5
2	2023-01-01 10:02:00	B	5.0
3	2023-01-01 10:03:00	A	12.3

Next steps: Generate code with df | New interactive sheet | Generate code with df_cleaned | New interactive sheet

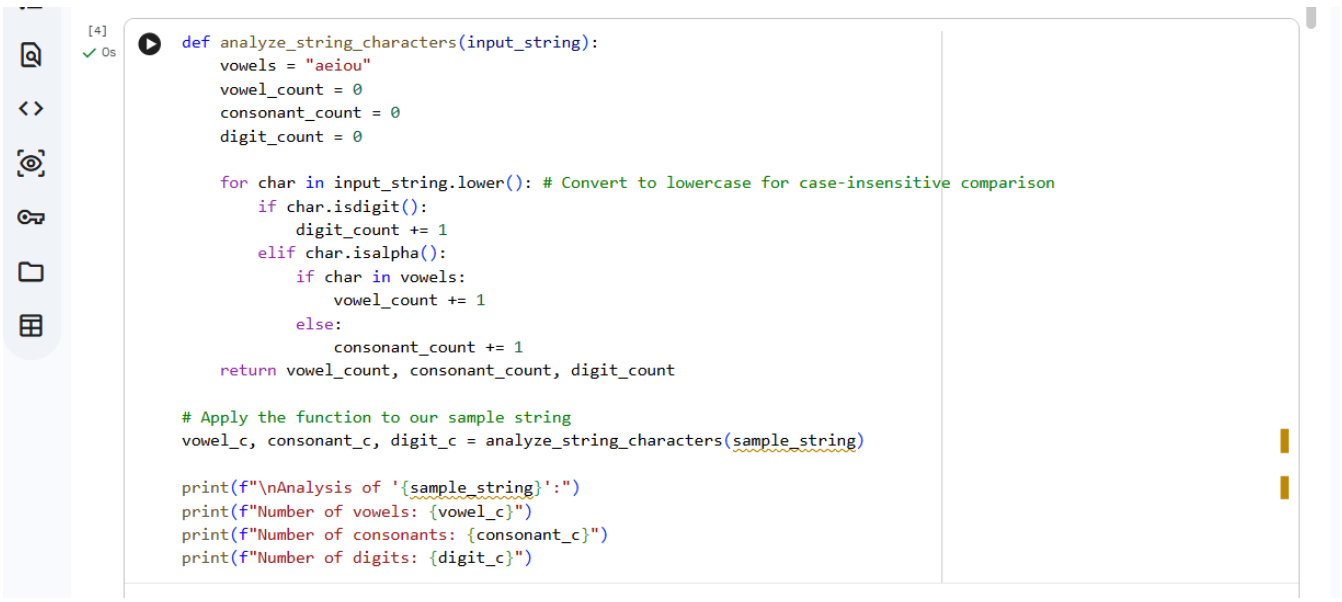
OBSERVATION:

The observation from this task is that the original list of sensor readings contains both valid and invalid values, where negative numbers represent incorrect or unusable sensor data. After executing the function generated using Gemini in Google Colab, all negative values are removed, and only valid, non-negative sensor readings remain in the output list. The comparison between the before and after lists clearly demonstrates that the cleaning logic works as intended. This confirms that AI-assisted code generation can efficiently handle basic data-cleaning tasks and helps ensure that the sensor data is accurate and suitable for further analysis.

TASK - 02 : String Character Analysis

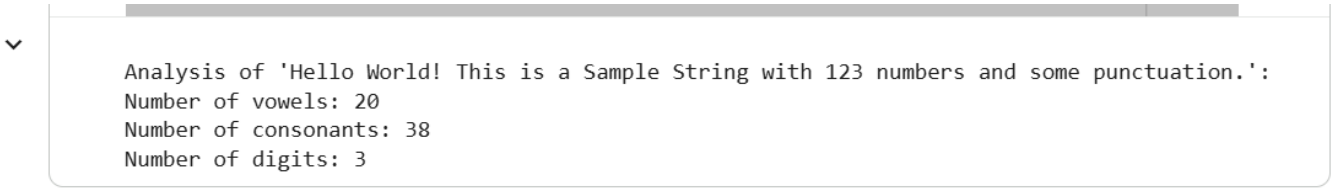
PROMPT : Use Gemini to generate a Python function that counts vowels, consonants, and digits in a string.

CODE :



```
[4]
✓ 0s
def analyze_string_characters(input_string):
    vowels = "aeiou"
    vowel_count = 0
    consonant_count = 0
    digit_count = 0

    for char in input_string.lower(): # Convert to lowercase for case-insensitive comparison
        if char.isdigit():
            digit_count += 1
        elif char.isalpha():
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1
    return vowel_count, consonant_count, digit_count

# Apply the function to our sample string
vowel_c, consonant_c, digit_c = analyze_string_characters(sample_string)

print(f"\nAnalysis of '{sample_string}':")
print(f"Number of vowels: {vowel_c}")
print(f"Number of consonants: {consonant_c}")
print(f"Number of digits: {digit_c}")
```

OUTPUT:



```
Analysis of 'Hello World! This is a Sample String with 123 numbers and some punctuation.':
Number of vowels: 20
Number of consonants: 38
Number of digits: 3
```

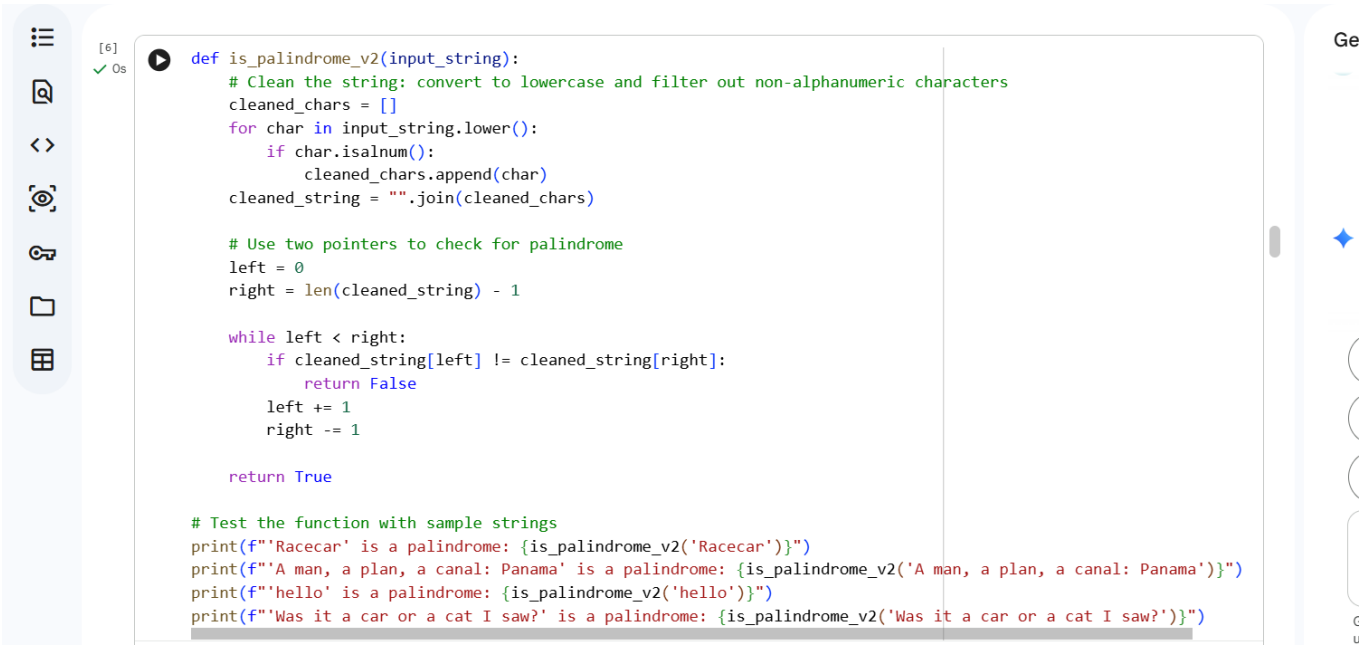
OBSERVATION:

The observation from this task is that the Python function generated using Gemini correctly analyzes the input string by iterating through each character and classifying it as a vowel, consonant, or digit. When tested with sample inputs, the function accurately counts vowels irrespective of case, correctly identifies consonants while ignoring special characters and spaces, and precisely counts numeric digits. The output matches the expected counts for all test cases, demonstrating that the function is reliable and effective for basic text analysis. This task also shows that Gemini can be effectively used to generate clean, logical, and reusable code for string processing in Python.

TASK – 03 : Palindrome Check – Tool Comparison

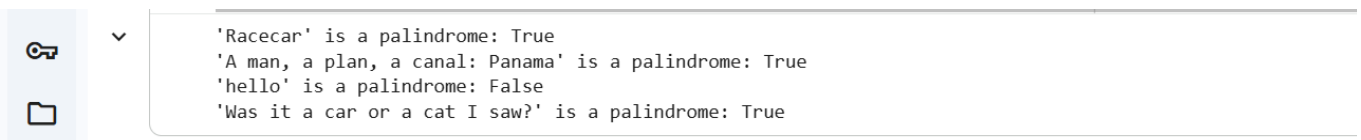
PROMPT: Generate two Python functions to check if a string is a palindrome, then compare them based on their clarity, conciseness, efficiency, and how they handle string logic (e.g., case sensitivity, non-alphanumeric characters), and finally provide a summary of the comparison highlighting which function offers clearer string logic.

CODE :



```
[6]
✓ 0s
def is_palindrome_v2(input_string):
    # Clean the string: convert to lowercase and filter out non-alphanumeric characters
    cleaned_chars = []
    for char in input_string.lower():
        if char.isalnum():
            cleaned_chars.append(char)
    cleaned_string = "".join(cleaned_chars)

    # Use two pointers to check for palindrome
    left = 0
    right = len(cleaned_string) - 1

    while left < right:
        if cleaned_string[left] != cleaned_string[right]:
            return False
        left += 1
        right -= 1

    return True

# Test the function with sample strings
print(f"'Racecar' is a palindrome: {is_palindrome_v2('Racecar')}")
print(f"'A man, a plan, a canal: Panama' is a palindrome: {is_palindrome_v2('A man, a plan, a canal: Panama')}")
print(f"'hello' is a palindrome: {is_palindrome_v2('hello')}")
print(f"'Was it a car or a cat I saw?' is a palindrome: {is_palindrome_v2('Was it a car or a cat I saw?')}")
```

OUTPUT:



```
'Racecar' is a palindrome: True
'A man, a plan, a canal: Panama' is a palindrome: True
'hello' is a palindrome: False
'Was it a car or a cat I saw?' is a palindrome: True
```

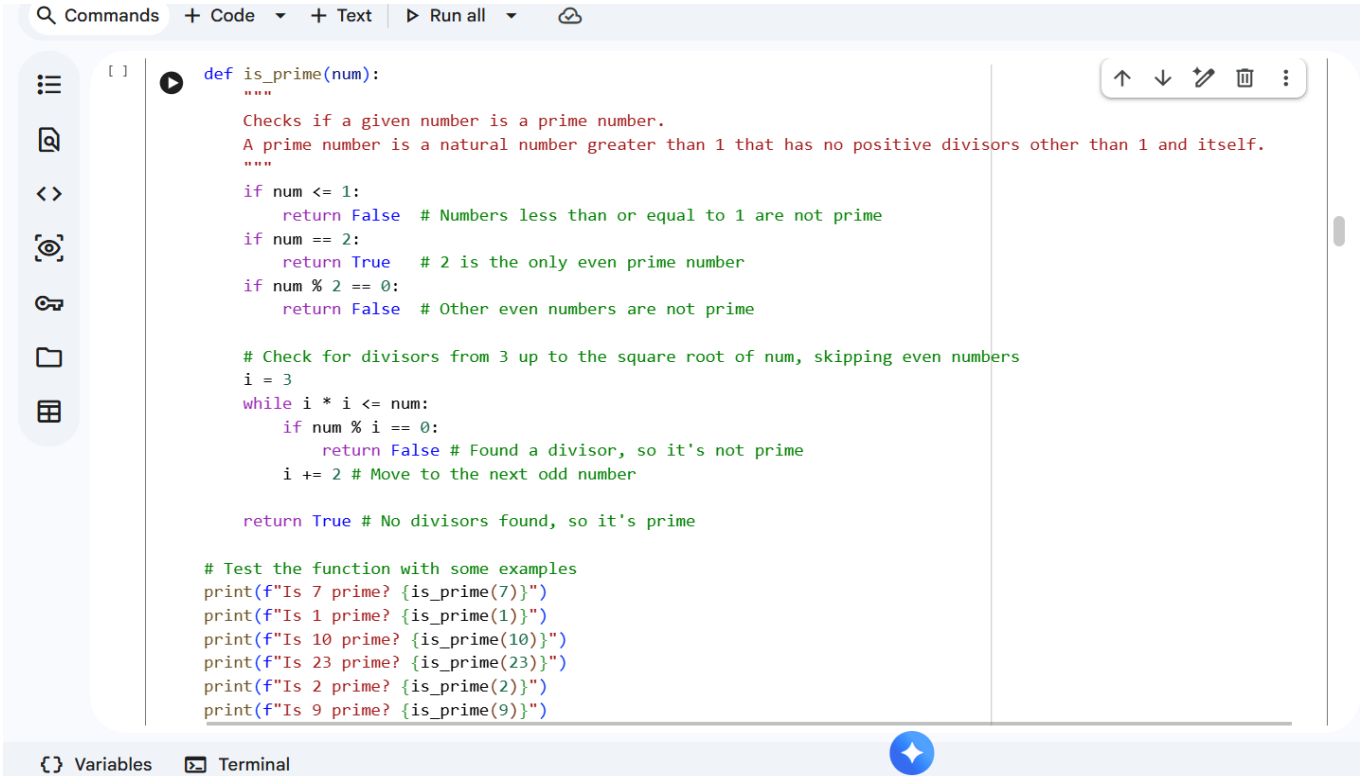
OBSERVATION:

The observation from this task is that both Gemini and Copilot generate correct palindrome-checking functions that accurately determine whether a given string reads the same forwards and backwards. However, the Gemini-generated solution tends to be more explicit and beginner-friendly, often including clear variable names and step-by-step logic that improves readability and understanding. In contrast, the Copilot-generated code is usually more concise and optimized, relying on Python's built-in features such as string slicing, which makes it efficient but slightly less descriptive for new learners. Overall, Gemini provides greater clarity and structured explanation of the logic, while Copilot favors brevity and developer efficiency, making each tool suitable for different levels of expertise and use cases.

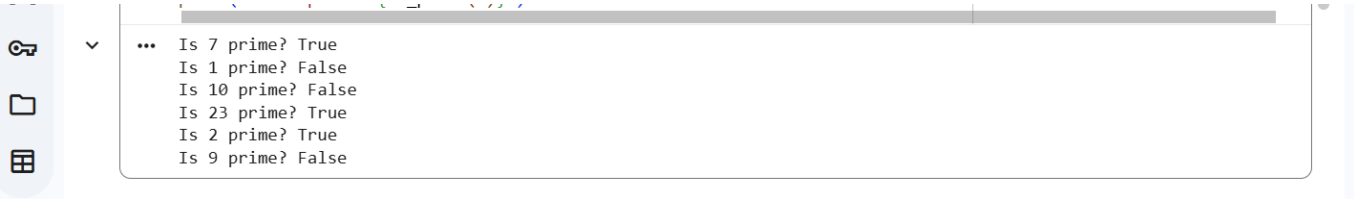
TASK-04 : Code Explanation Using AI

PROMPT: Ask Gemini to explain a Python function (prime check OR palindrome check) line by line

CODE :



OUTPUT :



OBSERVATION:

The observation from this task is that Gemini provides a clear and structured line-by-line explanation of the given Python function, breaking down each statement and describing its purpose in simple and understandable terms. The explanation helps bridge gaps in understanding by clarifying how conditions, loops, and return statements work together to achieve the desired result, whether checking for a prime number or a palindrome. As a result, the student is able to follow the logic more confidently, identify the flow of execution, and understand not just *what* the code does but *why* it works. This demonstrates that AI-based code explanation is highly effective for reviewing unfamiliar code and supports better learning and comprehension.